Frequent Pattern Growth Algorithm

The FP-Growth (Frequent Pattern Growth) algorithm efficiently mines frequent item sets from large transactional datasets. Unlike the Apriori algorithm which suffers from high computational cost due to candidate generation and multiple database scans. FP-Growth avoids these inefficiencies by compressing the data into an FP-Tree (Frequent Pattern Tree) and extracts patterns directly from it.

How FP-Growth Works

1. **Data Compression:** First FP-Growth compresses the dataset into a smaller structure called the Frequent Pattern Tree (FP-Tree). This tree stores information about item sets (collections of items) and their frequencies without need to generate candidate sets like Apriori does.
2. **Mining the Tree:** The algorithm then examines this tree to identify patterns that appear frequently based on a minimum support threshold. It does this by breaking the tree down into smaller "conditional" trees for each item making the process more efficient.
3. **Generating Patterns:** Once the tree is built and analyzed the algorithm generates the frequent patterns (itemsets) and the rules that describe relationships between items.

Working of FP- Growth Algorithm

Problem Statement: Consider a small grocery store transaction dataset. Each entry shows the set of items purchased together by a customer:

Transaction ID	Sorted Items by Frequency
T1	Bread, Milk, Butter
T2	Bread , Butter
T3	Bread, Milk
T4	Milk , Butter
T5	Bread

Apply the FP-Growth algorithm to identify frequent itemsets (groups of items frequently bought together), using a minimum support count of 2.

Step 1: Compute Item Frequencies

Scan the entire dataset one time to determine how often each item appears.

Item	Support Count
Milk	3
Bread	4
Butter	3

All items meet the minimum support threshold (≥ 2), so none are removed.

Step 2: Order Items in Each Transaction by Frequency

Next, sort every transaction in descending order based on how frequently the items appear overall.

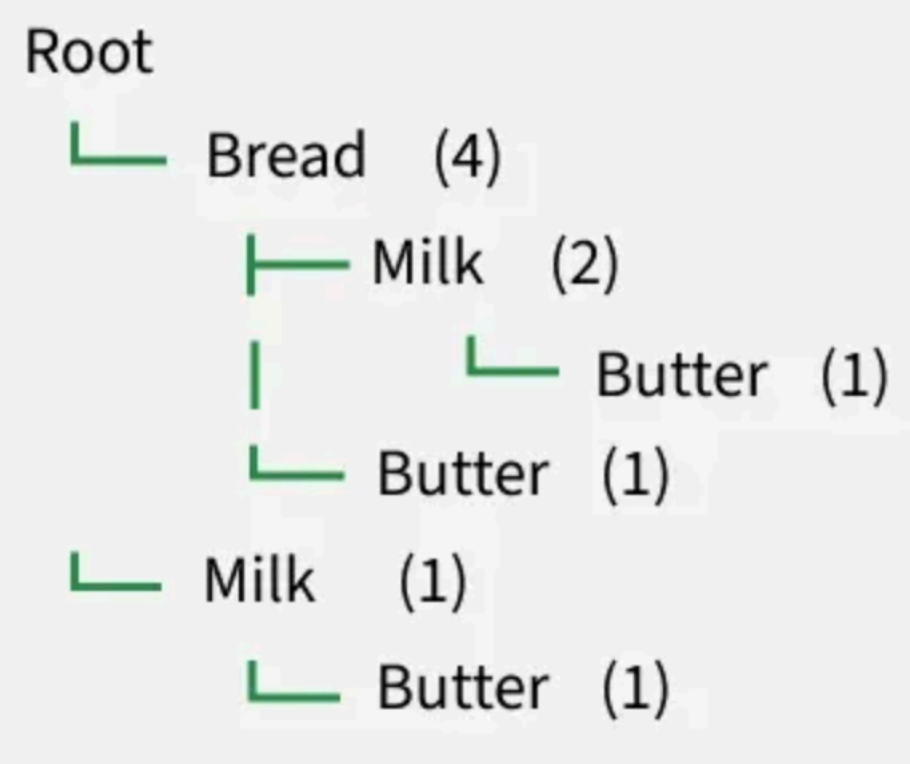
Transaction ID	Sorted Items by Frequency
T1	Bread, Milk, Butter
T2	Bread , Butter
T3	Bread, Milk
T4	Milk , Butter
T5	Bread

Step 3: Construct the FP-Tree

Insert the sorted transactions one by one into the FP-tree, merging any shared prefixes.

FP-Tree Insertions:

- **From T1:** Bread → Milk → Butter
- **From T2:** Bread → Butter
- **From T3:** Bread → Milk
- **From T4:** Milk → Butter (new branch since it doesn't begin with Bread)
- **From T5:** Bread



Step 4: Determine Conditional Pattern Bases

A conditional pattern base contains all prefix paths leading to a specific item. Let's examine the paths ending with Butter.

Paths that end with Butter:

- Bread → Milk → Butter (1 occurrence)
- Bread → Butter (1 occurrence)
- Milk → Butter (1 occurrence)

Thus, the conditional pattern base for Butter is:

[(Bread, Milk): 1, (Bread): 1, (Milk): 1]

Step 5: Build Conditional FP-Trees

Using the conditional pattern base, construct a smaller FP-tree for each item to identify frequent patterns involving that item. Butter's conditional FP-tree input:

- (Bread, Milk): 1
- (Bread): 1
- (Milk): 1

Count all items:

- Bread: 2
- Milk: 2

Since both meet the support threshold (≥2), we can now generate frequent patterns:

- {Butter, Bread}
- {Butter, Milk}
- {Butter, Bread, Milk}

Step 6: Extract All Frequent Itemsets

Frequent Itemset	Support Count
{Bread}	4
{Milk}	3
{Butter}	3
{Bread, Milk}	2
{Bread , Butter}	2
{Milk , Butter}	2
{Bread , Milk , Butter}	1 ✕ Not included (support < 2)